

habits in SQL.

What's happening

`FROM sales_reps r` assigns the shorthand `r` to the `sales_reps` table for the life of that query. From that point on, `r.rep_id` means "the `rep_id` column *from* `sales_reps`." Same idea with `agg` — it's the alias given to the subquery result.

Why it matters practically

1. Disambiguation — the real reason it exists

When two tables share a column name, the database engine *requires* you to specify which one you mean. Both `sales_reps` and `sales` likely have a `rep_id` column. Without a prefix, this is a runtime error:

```
sql
-- ❌ Ambiguous — which table's rep_id?
SELECT rep_id FROM sales_reps JOIN sales ON ...

-- ✅ Unambiguous
SELECT r.rep_id FROM sales_reps r JOIN sales s ON s.rep_id = r.rep_id
```

2. Readability at a glance

In a multi-table query, `r.name` tells any reader immediately: *this column belongs to the reps table, not the aggregation*. `agg.total_sales` tells you it's a computed column from the subquery — not a raw column from any base table. You understand data provenance without scrolling up to re-read the `FROM` clause.

3. Query brevity

Full table names get verbose fast, especially with real-world naming like `healthcare_billing.dbo.claim_line_item`. An alias collapses that to `cli`.
everywhere

Reply...



Sonnet 4.6 Extended

